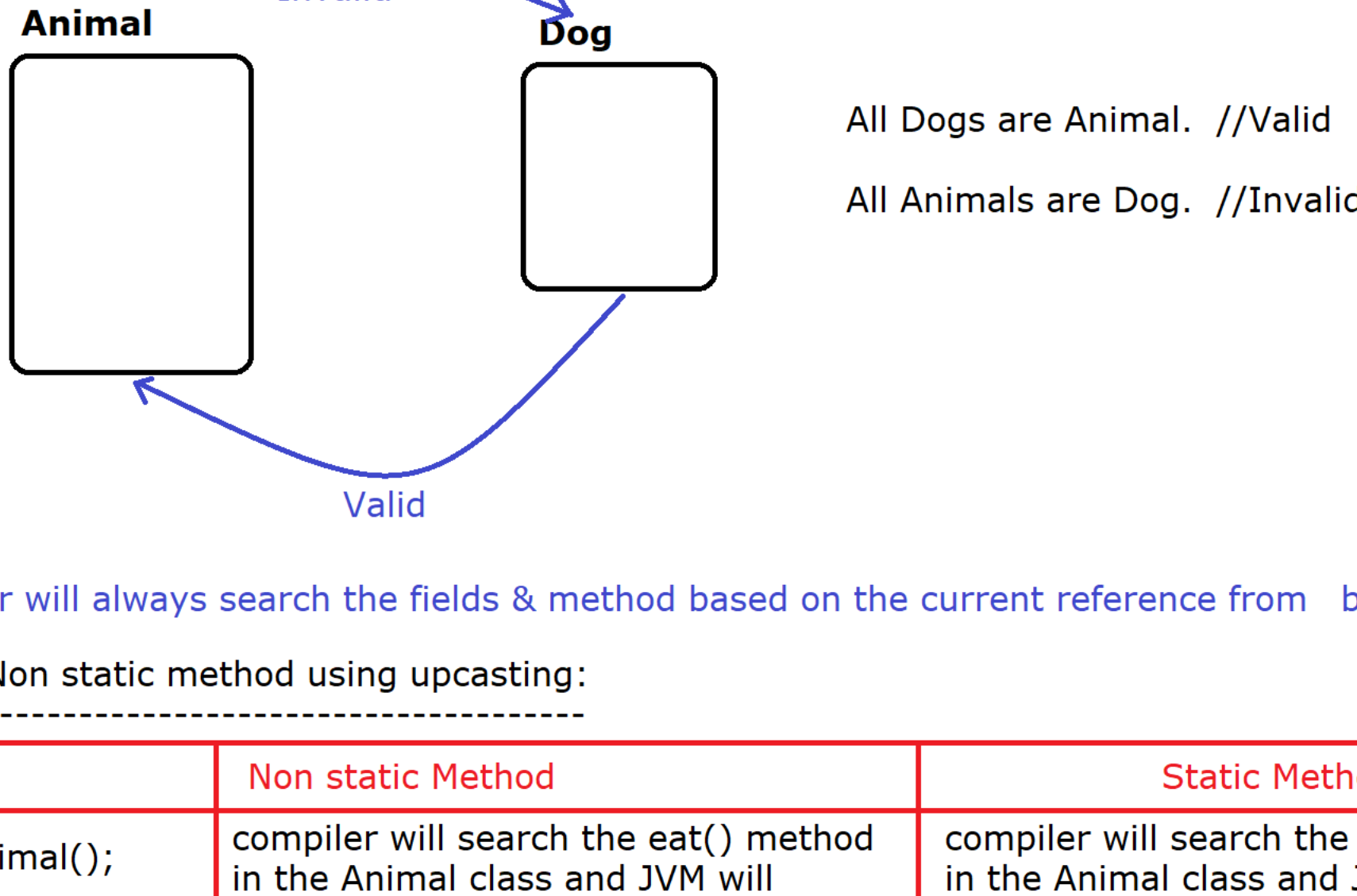


What is Up-casting and Downcasting ?

Upcasting :

* Assigning the **sub class object to the super class reference variable is called upcasting.**



Note : Java compiler will always search the fields & method based on the current reference from bottom to top

Calling the static & Non static method using upcasting:

Different Cases	Non static Method	Static Method
Animal a = new Animal(); a.eat();	compiler will search the eat() method in the Animal class and JVM will execute from Animal class. [Bottom to top]	compiler will search the eat() method in the Animal class and JVM will execute from Animal class. [Bottom to top]
Dog d = new Dog(); d.eat();	compiler will search the eat() method in the Dog class and JVM will execute from Dog class. [Bottom to top]	compiler will search the eat() method in the Dog class and JVM will execute from Dog class. [Bottom to top]
Animal a = new Dog(); a.eat(); (Upcasting)	compiler will search the eat() method from Animal class but JVM will start executing from Dog class. [Bottom to top]	compiler will search the eat() method in the Animal class and JVM will also execute from Animal class only because static methods are not overridden [Method Hiding]

```
//Program
class Animal
{
    public void eat()
    {
        IO.println("Generic Animal is eating");
    }
}
class Dog extends Animal
{
    public void eat()
    {
        IO.println("Dog Animal is eating");
    }
}
class Puppy extends Dog
{
    public void eat()
    {
        IO.println("Puppy Animal is eating");
    }
}

public class MethodOverriding
{
    public static void main(String[] args)
    {
        Animal a = new Puppy();
        a.eat();
    }
}
```

@Override Annotation :

- * It is a new feature introduced from JDK 1.5V onwards.
- * All the annotations in java must start with @ symbol. [Internally It is a Java class only]
- * It is optional but recommended that when we override a method from super class, we should use @Override annotation before the overridden method.
- * If we use @Override annotation and if the method is **not available** in the super class then we will get **compilation error.**

```
package com.ravi.method_overriding;

class Shape
{
    public void draw()
    {
        IO.println("Generic Drawing");
    }
}
class Circle extends Shape
{
    @Override
    public void draw()
    {
        IO.println("Drawing Circle");
    }
}
class Rectangle extends Shape
{
    @Override
    public void draw()
    {
        IO.println("Drawing Rectangle");
    }
}

public class MethodOverrdingDemo2
{
    public static void main(String[] args)
    {
        //Dynamic Method Dispatch
        Shape s = null;
        s = new Circle();      s.draw();
        s = new Rectangle();   s.draw();
    }
}
```

Variable Hiding concept with Upcasting :

```
class Super
{
    int x = 100;
}
class Sub extends Super
{
    int x = 200; //Variable Hiding
}
```

Only non static methods are overridden in java but not the variables[variables are not overridden in java] because behavior of an object (method) will change but not the property(variable).

Note :- In upcasting, non static fields will be always executed based on the current reference whereas methods are executed based on the object.

IN UPCASTING NON STATIC FIELDS ARE ALWAYS EXECUTED FROM CURRENT REFERENCE

```
package com.ravi.method_overriding;

class RBI
{
    protected String ifscCode = "RBIHYD000001";

    public String loan()
    {
        return "Bank should provide loan to customers";
    }
}
class SBI extends RBI
{
    protected String ifscCode = "SBIHYD000019"; //VH

    @Override
    public String loan()
    {
        return "Providing loan to customers @ 9.2% ROI";
    }
}

public class MethodOverridingDemo3 {

    public static void main(String[] args)
    {
        RBI rbi = new SBI();
        IO.println(rbi.ifscCode+" : "+rbi.loan());
    }
}
```

Can we override private methods of super class ?

No, We can't override private method of super class because private methods are not visible (not available) to the sub class hence we can't override.

We can't use @Override annotation on overridden method of sub class if super class method is declared with private access modifier because it is not overridden method, actually it can be re-declared by sub class developer.

```
package com.ravi.method_overriding;

class Alpha
{
    private void show()
    {
        IO.println("Private show method of Alpha class");
    }
}
class Beta extends Alpha
{
    public void show() //Re-declaration of super class method [Not overriding]
    {
        IO.println("Private show method of Alpha class");
    }
}
public class MethodOverridingDemo4
{
    public static void main(String[] args)
    {
        new Beta().show();
    }
}
```

Note :- private method of super class is not available or not inherited in the sub class so if the sub class declare the method with same signature then it is not overridden method, actually it is re-declared in the sub class.

Role of access modifier while overriding a method :

Access modifiers in Java : private -> default -> protected -> public

In terms of accessibility : public > protected > default > private

